

Vision-Based Vehicle Tracking using Particle Filtering

Jonathan Dumanski
Electrical Engineering
Stanford University
Stanford, USA
jdumansk@stanford.edu

Joana Mizrahi
Electrical Engineering
Stanford University
Stanford, USA
jmizrahi@stanford.edu

Abstract—This project assesses the performance of vehicle tracking using particle filtering with vision-only measurements. It also compares the effects of using different likelihood metrics for particle weighting, and which combinations of them are most effective for tracking. Our pipeline was evaluated on the KITTI Object Tracking dataset, which contains a series of stereo images and onboard GPS/IMU data. Constant Turn Rate and Velocity (CTRV) vehicle dynamics were chosen to model the vehicles being tracked, which is a simple yet well-representative model for basic vehicle motion. Our primary goal was to understand the benefits and drawbacks of using vision-only measurements for particle filtering, as well as which likelihood models provide the best tracking performance.

Index Terms—Vehicle tracking, Bayesian filtering, Stereo vision, CTRV model

I. INTRODUCTION

Autonomous vehicles require robust tracking of surrounding vehicles, particularly in GPS-denied environments. Vision-based tracking using stereo cameras offers a practical approach for estimating 3D positions of nearby vehicles [1]. However, the choice of Bayesian filter can significantly impact tracking performance when dealing with nonlinear vehicle dynamics, real-world measurement noise, and potentially non-Gaussian state distributions.

This project investigates two key questions: (1) How well can particle filtering track vehicle position and heading using only vision-based measurements? (2) Which combinations of likelihood models lead to the best tracking performance?

II. RELATED WORK

A. Stereo Vision for Object Tracking

The KITTI Vision Benchmark Suite provides standardized stereo camera sequences with ground truth vehicle trajectories, which enables quantitative evaluation of tracking algorithms. It has been used for evaluating autonomous driving perception algorithms [1].

Stereo vision is used to obtain direct 3D position measurements without requiring LiDAR, making it practical for cost-sensitive autonomous driving applications. Lim et al. demonstrated that Extended Kalman Filter combined with accurate disparity estimation can effectively track obstacle position and velocity, though measurement errors increase at longer ranges due to disparity quantization and matching errors

[2]. Their work emphasized that accurate disparity estimation is critical for precise distance measurement, motivating the use of robust stereo matching algorithms.

Stereo-based tracking faces several challenges. Occlusions in cluttered urban scenes can cause missed detections and tracking failures. Lighting variations from shadows, reflections, and changing weather conditions affect feature matching quality. Additionally, disparity errors grow quadratically with distance, degrading depth accuracy for far vehicles [3]. These practical limitations motivate comparative evaluation of different filtering approaches under realistic measurement noise conditions.

B. CTRV Model for Vehicle Tracking

The selection of the motion model critically impacts tracking performance. The Constant Turn Rate and Velocity (CTRV) model explicitly represents turn rate as a state variable, enabling accurate modeling of curvilinear motion common in urban driving scenarios.

Schubert et al. conducted comprehensive comparisons of motion models including CV, CA, CTRV, and Constant Turn Rate and Acceleration (CTRA) for vehicle tracking [4]. Their experimental results on real-world data demonstrated that CTRV significantly outperforms simpler linear models during turning maneuvers while maintaining comparable performance during straight-line motion.

C. Bayesian Filtering for Object Tracking

Bayesian methods for vision-based object tracking are widely used in literature, built upon common filter frameworks such as the EKF, UKF, and particle filter. Due to its flexibility in being able to handle non-linear dynamics and non-Gaussianity in both the posterior and noise, the particle filter and variations of it have been widely used to solve the estimation problem from vision measurements. Several survey papers and articles document the use of variations of the particle filter for vision-based tracking over Gaussian-based approaches, mainly due to the relaxation of non-Gaussian distributions [7] [8].

The particle filter also has the flexibility of being able to explicitly model likelihood functions of multiple independent measurement sources, which Gaussian-based filters cannot

model very well when the likelihood exhibits significant non-Gaussianity. For image measurements, different features of image data can be used to develop independent likelihood models that can be chained to generate a more robust and accurate overall likelihood. For vision-based measurements, common likelihood models are based on depth and reprojection error to measure the discrepancy between the calculated image coordinates of the object from measurements, and the reprojected pixel coordinates of particles from their 3D position. Other models based on other image features have also been developed and used, such as color-based models [9]–[12], which typically measure the distance between particle color histograms and a target color histogram using the Bhattacharya distance. Other image features, such as texture [12], shape [10], and edges [11], have also been used. This approach is quite powerful as it allows for multiple independent image features to be used in order to gain a better estimate of the true likelihood function.

Although limited by Gaussian state distributions and noise, Gaussian-based filters have also been used in vision-based tracking applications. The IMM-EKF has been used for tracking mobile targets from a UAV [6] using a switchable dynamics approach, resulting in robust tracking performance.

III. PROBLEM FORMULATION AND APPROACH

A. Dataset

For model inference, validation and testing we used the KITTI Object Tracking dataset, which contains stereo camera sequences of urban driving. This dataset provided rectified left and right stereo images, onboard GPS and IMU data of the camera vehicle, and ground truth values of the position, heading, and bounding box dimensions of vehicles at each frame.

The stereo images are used as the input to our pipeline at each timestep as the filter runs, while the GPS and IMU data are only used to define and compute the pose transformations between camera frames. The ground truth data is used for both model evaluation (running experiments to analyze tracking performance under different likelihood models) as well as final testing.

B. Reference Frames

We define the global reference frame be the reference frame of the camera at $t = 0$, and define the states of the vehicles being tracked with respect to this global frame. Therefore, at each timestep, we transform the measurements from the current camera frame at t to the global frame before performing any calculations.

The given GPS and IMU data include the latitude, longitude, altitude, roll, pitch, and yaw of the IMU at each timestep. We convert the latitude and longitude to metric units in the world frame using the Mercator projection formula, which gives us the pose of the camera (technically the pose of the IMU, however KITTI provides the fixed transformation matrix from IMU to camera) in the world frame at each timestep. We then calculate the relative pose transformations from the

camera at each timestep to the global reference frame (the camera frame at $t = 0$), which we require for transforming local measurements to the global frame, as mentioned.

We also follow the coordinate system convention imposed by KITTI, which is a right hand coordinate system oriented such that the z axis points forward, the x axis points to the right, and the y axis points downward. It also defines that the y rotation angle (heading angle) is 0 along the x axis (and counter-clockwise positive about the y axis since it's a right hand system).

C. System Architecture

The tracking pipeline can be segmented into 2 stages: the front end and the back end. The front end deals with data collection, processing, and refinement, which converts the raw measurements into structured inputs for the particle filter. The outputs of the front end are the mask of the vehicle being tracked in the left image, M_t , and the depth map of the left image, D_t .

The mask is obtained by running the left stereo image through the YOLO26-seg model, which produces a list of masks of all of the vehicles in the image. The mask corresponding to the vehicle we are tracking then chosen from this list using methods discussed in the following section.

The depth map is obtained by running both the left and right images through OpenCV's Semi Global Block Matching function, which produces a disparity map of the left image. This disparity map can be converted directly to a depth map using camera intrinsics, which will be elaborated on in the following section.

Both of these will be formulated in detail at the end of section 3, since they use techniques closely related to the likelihood models.

The backend simply consists of the particle filter, which is comprised of the transition (dynamics) model for particle propagation, and the likelihood model for weight updates.

Figure 1, shows the overall system architecture and high-level components.

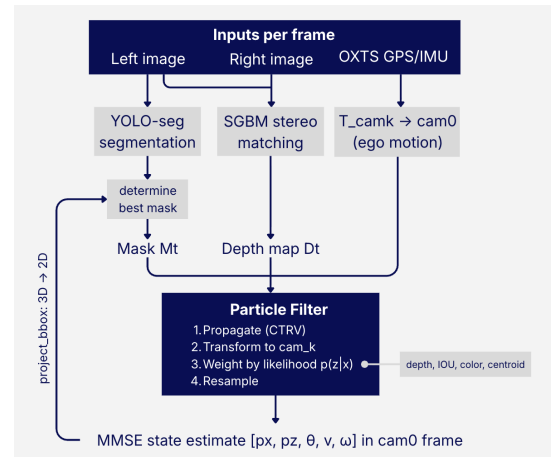


Fig. 1: Vehicle tracking system architecture.

D. Vehicle Dynamics Model

We chose to model for the vehicle being tracked using the state $\mathbf{x}_k = [x_k, y_k, \theta_k, v_k, \omega_k]^T$, representing 2D position, heading angle, velocity magnitude, and turn rate. With this state, we chose to model the vehicle dynamics using the CTRV kinematic model. Our measurements mainly provide information about vehicle pose, and minimal information about higher derivatives of pose components such as velocity and turn rate. Therefore, we deemed the CTRV model an appropriate choice since it models planar position and heading angle explicitly, and models changes in velocity and turn rate implicitly through noise propagation, which align well with the kinematic information we obtain from our measurements. Additionally, the model has been proven to perform well for vehicle tracking in general since it captures realistic vehicle motion which are typically characterized by approximately constant velocity and turn rate over short time intervals.

The discretized CTRV model dynamics are described in a piecewise manner, depending on the magnitude of the turn rate, ω_k :

Case 1: $|\omega| > \epsilon$ (sufficient turn rate)

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \\ v_{k+1} \\ \omega_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + \frac{v_k}{\omega_k} [\sin(\theta_k + \omega_k \Delta t) - \sin(\theta_k)] \\ y_k + \frac{v_k}{\omega_k} [-\cos(\theta_k + \omega_k \Delta t) + \cos(\theta_k)] \\ \theta_k + \omega_k \Delta t \\ v_k + \eta_v \\ \omega_k + \eta_\omega \end{bmatrix}$$

Case 2: $|\omega| < \epsilon$ (approximately linear motion)

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \\ v_{k+1} \\ \omega_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + v_k \Delta t \cos(\theta_k) \\ y_k + v_k \Delta t \sin(\theta_k) \\ \theta_k \\ v_k + \eta_v \\ \omega_k + \eta_\omega \end{bmatrix}$$

E. Likelihood Modeling

The likelihood model in the particle filter is what updates the weight of each particle, where each weight corresponds to the likelihood of the observed measurements given the particle's state at the current timestep.

Recall, our measurements from the front end are $z_t = [M_t, D_t]^T$, where M_t is the predicted mask of the vehicle from YOLO, and D_t is the depth map of the left image obtained from OpenCV. The likelihood of the measurements given a particles state, x_i , is expressed as $p(z_t|x_i) = p(M_t, D_t|x_i)$.

The likelihood model was constructed using multiple individual likelihood terms, each corresponding to a different measurement cue related to the true vehicle state.

1) *Mask IoU Likelihood*: The first likelihood term used was the mask IoU likelihood. This term is calculated by taking the IoU (intersection over union) of the measurement mask, M_t , with the convex hull mask of the given particle, $M_{cvx_{x_i}}$. The convex hull mask is computed by taking the particle, forming a 3D bounding box around it, projecting the vertices of this bounding box into camera coordinates, calculating the convex

hull of these points, and forming a mask of the interior of the convex hull. This process is visualized in Figures 2 and 3



Fig. 2: Vehicle 3D bounding box.



Fig. 3: Vehicle convex hull mask.

The projection of 3D bounding box points to 2D camera points is performed using the camera calibration matrix, $P = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$ (provided by KITTI), which contains intrinsic and extrinsic parameters of the camera assuming a standard pinhole camera model. Also note the projection is done in homogeneous coordinates.

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

R and t represent the pose transformation from the reference frame the 3D points are in to camera frame, however since the 3D points are assumed to be in the camera frame already, the transformation just becomes identity ($R = I$, $t = 0$) (this will be reiterated in the Particle Filter section, but all particles are transformed into the current camera frame before being passed into the likelihood function using the camera pose transformation matrices mentioned in the Reference Frames section).

Within K , f_x and f_y are the focal lengths (in pixels) in the x and y directions, and c_x and c_y are the x and y coordinates of the principal point (location where the optical axis intersects the image plane).

Given 2 masks, the IoU is defined as:

$$IoU(M_1, M_2) = \frac{|M_1 \cap M_2|}{|M_1 \cup M_2|}$$

The raw IoU function output is bounded between 0 and 1, however in order to provide a stronger discrimination between good and bad particles, we use the following likelihood formulation for the mask IoU likelihood:

$$p_{IoU}(M_t|x_i) = \exp(\alpha(IoU(M_t, M_{cvx_{x_i}}) - 1))$$

Where a higher alpha means a tighter likelihood distribution (only particles with very high IoU scores survive). Based on empirical testing a value of $\alpha = 5$ was selected.

The intuition behind this likelihood metric, is that assuming the measurement mask M_t is indeed the mask of the vehicle we are tracking, the convex hull mask of the true vehicle should overlap with M_t very well.

2) *Depth Likelihood*: For a given particle, the depth likelihood is calculated by comparing the 10th percentile measured depth of the masked depth map with the depth (z) value of the closest vertex of the 3D bounding box of the particle. Intuitively, we are comparing the depth of the closest point on the vehicle calculated based on the measurements vs the given particle state. The measurement depth is calculated using the 10th percentile depth value for robustness to outliers, since the depth map produced by OpenCV's SGBM implementation can exhibit high local variance and outlier disparities.

Once the measurement depth, Z_{meas} , and particle depth, $Z_{pred_{x_i}}$ values are calculated, the likelihood is calculated as:

$$p_{depth}(D_t|x_i) = \exp\left(-\frac{(Z_{meas} - Z_{pred_{x_i}})^2}{2\sigma_Z^2}\right)$$

A σ_Z value of 2m was selected based on empirical results.

3) *Color Likelihood*: Recall from the mask IoU likelihood section, that we calculate a convex hull mask of each particle, representing the estimate 2D projection of the vehicle at that particle state into camera coordinates. Given a particle, x_i , the color likelihood is calculated by comparing the color distributions (histograms) of the pixels within the measurement mask, M_t , and the convex hull mask of the particle, $M_{cvx_{x_i}}$. Before computing the histograms, the masked regions are converted from RGB to HSV (hue-saturation-value) color space. The histograms are then computed only using the hue and saturation components of each pixel. The reason for this is that unlike the RGB representation of color, HSV separates the actual color (represented by hue and saturation) from the illumination (brightness) of that color (represented by the value component). Therefore by only comparing the distribution of hue and saturation between the 2 masked regions, our similarity metric becomes illumination invariant, which is quite useful in circumstances where the perceived brightness of the vehicle being tracked changes from one frame to the next (for example, if it goes from being in the sun to entering a dark tunnel).

Once the 2 hue-saturation histograms are calculated for each of the masks, they are compared using the Bhattacharyya distance, which is a metric for comparing the similarity between 2 discrete probability distributions (which the color histograms can be normalized and interpreted as). Given 2 discrete probability distributions, the Bhattacharyya distance is given as

$$D_B(P, Q) = -\ln\left(\sum_{x \in X} \sqrt{P(x)Q(x)}\right)$$

With this, the color likelihood of a measurement given the a particle x_i is calculated as:

$$p_{color}(M_t|x_i) = \exp(-cB_d(H_{M_t}, H_{M_{cvx_{x_i}}}))$$

A value of $c = 10$ was selected based on empirical results.

Algorithm 1 Sequential Importance Resampling Particle Filter

0: **for** $t = 1, 2, \dots$ **do**

0: **for** $i = 1, \dots, N$ **do**

0: Sample a predicted particle:

$$x_{t+1}^i \sim p(x_{t+1} | x_t^i)$$

0: Compute the importance weight:

$$w_t^i = p(z_t | x_t^i)$$

0: **end for**

0: Normalize weights $\{w_t^i\}_{i=1}^N$

0: Compute effective sample size

$$ESS = \frac{1}{\sum_{i=1}^N (w_t^i)^2}$$

0: **if** $ESS < 0.5 N$ **then**

0: **for** $i = 1, \dots, N$ **do**

0: Draw index $j \in \{1, \dots, N\}$ with probability proportional to w_t^j

0: Set $\bar{x}_t^i = x_t^j$

0: **end for**

0: Set $\mathcal{X}_t = \{\bar{x}_t^i\}_{i=1}^N$

0: **end if**

0: **end for=0**

4) *Centroid Likelihood*: Given a particle x_i , the centroid likelihood is calculated by comparing the centroid of the measurement mask, M_t to the geometric center of the 3D bounding box of the particle projected into camera coordinates. The centroid likelihood is expressed as:

$$p_{centroid}(M_t, x_i) = \exp\left(-\frac{\|c_{M_t} - c_{geo_{x_i}}\|^2}{\sigma_c^2}\right)$$

A σ_c value of 2m was selected based on empirical results.

With these 4 likelihood models, the overall likelihood is computed as the product of the 4 individual likelihoods:

$$p(z_t|x_i) = p_{IoU}(M_t|x_i)p_{depth}(D_t|x_i)p_{color}(M_t|x_i)p_{centroid}(M_t, x_i)$$

Note that we use log-weights in the implementation for numerical stability.

F. Filter Implementation

As mentioned in the System Architecture section, the backend uses a particle filter to process the front end measurements (M_t and D_t) and update posterior distribution of the vehicle state. The particle filter algorithm used for vehicle tracking is given in Algorithm 1.

This implementation of the particle filter includes importance resampling, which only resamples particles when the effective sample size (ESS) of the current state particle distribution goes below some threshold. The ESS is used quantity the degree of particle degeneracy, which is the degree to which the mass of the weights collapses to a very small number of particles (where a smaller ESS indicates a higher concentration of mass on a fewer number of particles). High

particle degeneracy (low ESS) can lead to unstable tracking and filter divergence due to most of the particles having effectively no contribution to the posterior, hence we resample when the ESS becomes sufficiently low. A common threshold of $0.5N$ [13] (where N is the number particles in the filter), which was employed in our implementation.

Another important detail was the number of particles chosen for the filter. In general, a safe number of particles to have is 10^n , where n is the dimension of the state vector. In our case, $n = 5$, however, upon inference, this resulted in impractical runtime of up 10's of seconds for one iteration of the particle filter, since each particle must be propagated through the dynamics, and then have a likelihood weight calculated for it using the 4 individual likelihood functions. With this, we drastically reduced the number of particles and found that a distribution size of $N = 500$ particles was sufficient to accurately represent the state distribution and maintain a practical runtime.

The particle filter was chosen over Gaussian based filters such as the EKF or UKF since it is able to deal with both non linear dynamics and non-Gaussian distributions over the state (Gaussian filters can deal with non linear dynamics, but require the state to be normally distributed). Some of the likelihood models from the section above can result in multimodal likelihood distributions over the state, which made the particle filter a natural choice. For example, if we have 2 white cars at the same depth but different x positions, both of these cars will have very similar color distributions and depth values, meaning both the color and depth likelihood distributions, just in x and z , would have 2 modes around the (x, z) positions of each car.

G. Measurement Acquisition

As mentioned in the System Architecture section, the 2 structured measurements used in the particle filter likelihood model are the the mask of the vehicle being tracked in the left image, M_t , and the depth map of the left image, D_t . Examples of the vehicle mask and depth map are shown in Figures 5 and 6. The vehicle mask is obtained by running the



Fig. 4: Left stereo image.



Fig. 5: Vehicle YOLO mask.

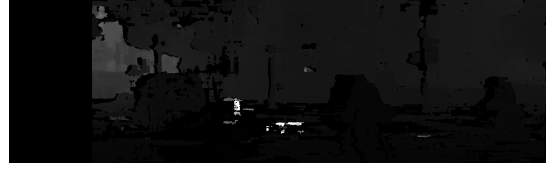


Fig. 6: Vehicle depth map.

left stereo image through the YOLO26-seg model, which, as mentioned above, returns a list of objects in the scene, each with a class label and mask.

To select the mask corresponding to the vehicle being tracked, we use information about the current estimate of the state (which can be queried from the particle filter), as well as information about the previous mask. Since we use information about the previous mask and current state, this implies that we must have a good estimate of the initial state and mask at $t = 0$. Before tracking a vehicle, we of course must know what vehicle we are tracking. Therefore, we manually identify the vehicle in the first frame (at $t = 0$) that we will be tracking, and use the ground truth position with minimal noise as our prior state estimate at ($t = 0$).

Given the lists of masks and class labels from at a given time $t = k$, we first filter out all the non-vehicles related classes. For each remaining vehicle, we simply calculate a score associated with the likelihood of a given mask being the mask of the vehicle we are tracking. To do so, we employ similar techniques to those used the likelihood functions for particle weighting. Before calculating the scores for each candidate vehicle mask in the list, we first generate the convex hull mask of the prior state estimate at the current time step by querying the MAP estimate from the particle filter. Note we use the MAP estimate since the posterior may be multimodal, for which the MMSE estimate would not give a representative point estimate of the true state. Then, we iterate through the candidate masks, and for each candidate mask, a score is computed as a linear combination of the following terms:

- 1) The IoU between the candidate mask and the prior convex hull mask.
- 2) The normalized squared distance between the centroids of the two masks.
- 3) The color histogram similarity between the candidate mask and the reference mask.
- 4) The 10th percentile depth similarity between the current mask and the reference depth.
- 5) The area similarity between the current mask area and reference area.
- 6) The IoU between the candidate mask and the previous measurement mask M_{t-1}

The reference terms above are calculated using the the mask from the previous iteration (with the intuition being that the mask should not change very much from one frame to the next since the frame rate is 10Hz).

The depth map is obtained using OpenCV's implementation of SGBM (semi-global block matching). This operation takes

in 2 rectified stereo images, and outputs a disparity map of the left image. A disparity map is an image where each value represents the horizontal displacement (disparity) between where that pixel appears in the left image and the right image ($d = x_l - x_r$). SGBM works reasonably well in feature rich regions, however, its performance tends to degrade in textureless, feature-lacking environments, or situations where an obstacle is occluded in one image but appears in the other. This occurs since in these situations, the matching of pixels between the left and right images can become ambiguous (or even impossible in the case of object occlusion). Fortunately, we only care about the masked depth map, which usually contains a vehicle, and vehicles often contain an sufficient number of features (distinct edges, corners, and other sudden changes in pixel intensity) for adequate stereo matching and disparity recovery.

Once the disparity map is computed, the depth map can be recovered using the depth-disparity relationship under the pinhole camera model $Z = \frac{Bf_x}{d}$, where B is the stereo baseline and f_x is the x-component of the focal length.

IV. RESULTS

This section will discuss experiments performed to evaluate likelihood model performance, as well as the overall systems performance on driving sequences from the KITTI dataset.

A. Ablation Study for Likelihood Model Selection

In order to evaluate the contributions of each of the individual likelihood models to the overall likelihood and analyze their affects on tracking performance, we performed an ablation study. We started with the 4 initial likelihood models mentioned in the likelihood section, and compared the tracking performance of the system using all of the likelihood models, as well as the performance when one model was left out. We performed this study on multiple driving sequences from the KITTI dataset. An example of the results obtained for one of the sequences is given in Figure 7. The accompanying RMSE values over the entire trajectory are displayed in Table I.

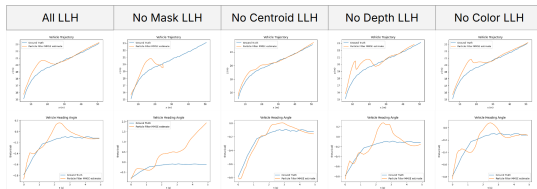


Fig. 7: Tracking performance under different likelihood model combinations.

From this study, we observed that the mask IOU, depth, and color likelihoods all increase tracking performance with respect to position and heading, which can be inferred by the fact that tracking performance decrease when they are removed from the overall likelihood. Interestingly, the tracking performance improves when the centroid likelihood is removed, indicating it actually hurts tracking performance. After further investigation, we found that adding the centroid likelihood

LLH Removed	x RMSE	z RMSE	Planar RMSE	Heading RMSE
None	0.717	0.801	1.075	0.121
Mask	10.340	1.451	10.441	0.903
Centroid	0.623	0.312	0.696	0.069
Depth	1.592	1.081	1.924	0.170
Color	0.744	0.760	1.063	0.107

TABLE I: Ablation study showing the effect of removing each likelihood term on tracking error. RMSE values are reported in meters for position and radians for heading.

improves performance when the back of the vehicle being tracked is square to the camera, and decreases performance when the vehicle is turning with respect to the camera. We concluded the reason for this was that the centroid likelihood compares the centroid of the measurement mask M_t to the geometric center of the 3D bounding box of a given particle. When the vehicle's rear is square to the camera, these 2 quantities align, so the comparison metric is valid, and thus improves performance. However, during turning sequences, the centroid of the mask lags behind the geometric center of the vehicle due to the front of the car appearing smaller as a consequence of perspective, resulting in the centroid of the mask shifting towards the back of the car. Therefore, the likelihood term favors particles whose geometric centers are close to the mask centroid, meaning it favors particles whose geometric centers lag behind the true geometric center, resulting in the filter "understeering" which is exactly what we observe in the plot above where the all likelihoods are present (compared to the plot where the centroid likelihood is removed). Therefore, in the final system model, we ended up removing from the centroid term from the likelihood function.

B. Overall System Evaluation and Performance

This section will go over the general results obtained from running our pipelines through several driving sequences from the KITTI dataset. Both general results and specific challenges will be discussed.

1) *General Results:* For most KITTI driving sequences, our finalized pipeline (which excludes the centroid likelihood) performed quite well at tracking both planar (x-z) position and heading angle. The planar position and heading angle of 4 longer driving sequences with both straight and turning segments along the trajectory are shown in Figure 8, overlaid with the ground truth KITTI trajectories. The corresponding RMSE values are shown in Table II

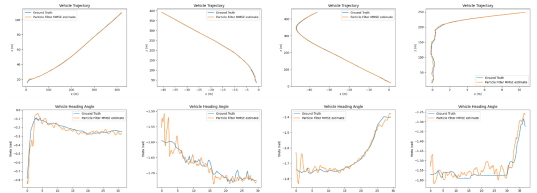


Fig. 8: Vehicle tracking performance for various driving sequences.

Sequence	x RMSE	z RMSE	Planar RMSE	Heading RMSE
0004	2.040	0.796	2.180	0.0505
0005	0.202	1.722	1.734	0.0219
0010	0.203	0.840	0.865	0.0258
0011	0.134	0.797	0.808	0.0348

TABLE II: Tracking error per KITTI sequence. RMSE values are reported in meters for position and radians for heading.

As can be seen, the both the position and angular tracking errors are quite low. The worst performing sequence has a position RMSE just exceeding 2m and the heading RMSE just exceeding 0.0505 rad over the entire sequence, however, this is acceptable considering the entire trajectory is over 400m.

2) *Challenges*: The particle filter behaved quite well when the measurements were reasonable, however tracking performance degraded when measurements (specifically the YOLO mask) were poor or incorrect. If our mask selection metric was not able to select the mask of vehicle we were tracking or YOLO was not able to identify the vehicle in the first place, the measurement mask would often be of some other vehicle we were not tracking, resulting in very poor tracking. We observed this occurred in the following situations:

- 1) Significant glare (affecting color detection, however usually only an issue when other vehicles were nearby, as this caused vehicle swapping).
- 2) Vehicle occlusion (severe partial and total occlusion, causing YOLO to either not identify the vehicle in the first place, or resulting in our metric having higher confidence in another vehicle).
- 3) Similar vehicles (other vehicles in the scene similar in color and planar location).

An example of severe occlusion is shown in Figure 9 where the white van being tracked is heavily occluded by the orange van in front, resulting in mask switching.

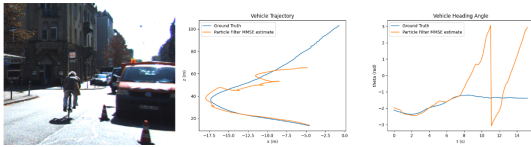


Fig. 9: Vehicle tracking performance during vehicle occlusion

V. CONCLUSION

This project presented a vision-based vehicle tracking pipeline using a particle filter with stereo depth and image-based likelihood cues. By combining a CTRV dynamics model with mask IoU, depth, and color likelihood terms, the proposed system was able to track vehicle planar position and heading with low error across multiple KITTI driving sequences using only camera measurements.

The ablation study showed that the mask IoU likelihood was the most important term for stable tracking, while the depth and color likelihoods also improved performance. In contrast, the centroid likelihood degraded performance during turning

maneuvers due to the mismatch between the mask centroid and the projected geometric center of the vehicle under perspective effects.

Overall, the results demonstrate that particle filtering is a strong framework for vision-based vehicle tracking because it can naturally handle nonlinear vehicle dynamics and non-Gaussian, multimodal likelihoods. At the same time, the experiments highlighted that the quality of the front-end measurements remains critical, as tracking performance degrades when mask detection or mask association fails.

VI. FUTURE WORK

As mentioned throughout the report, there were some challenges and limitations of our approach. With this, future work could include any of the following:

- 1) More advanced approaches for mask selection (such as learning based methods).
- 2) Testing on sequences with a greater variety of vehicle movement to fully evaluate robustness of our tracker.
- 3) Developing confidence metrics to detect unreliable mask and depth measurements and fall back on the vehicle dynamics model when necessary.

REFERENCES

- [1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite", in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [2] Y. C. Lim and K. Huh, "Improvement of stereo vision-based position and velocity estimation and tracking using a stripe-based disparity estimation and inverse perspective map-based extended Kalman filter", *Optics and Lasers in Engineering*, vol. 48, issue 9, pp. 858-869, 2010.
- [3] H. Hirschmüller, "Stereo processing by semiglobal matching and mutual information", *IEEE Trans. Pattern Analysis and Machine Intelligence*, 2008.
- [4] R. Schubert, E. Richter, and G. Wanielik, "Comparison and evaluation of advanced motion models for vehicle tracking," in *11th Int. Conf. on Information Fusion*, 2008.
- [5] "Performance evaluation of a maneuver classification algorithm using different motion models in a multi-model framework," *Sensors*, vol. 22, no. 1, 2022.
- [6] H. Cheng, L. Lin, Z. Zheng, Y. Guan, and Z. Liu, "An autonomous vision-based target tracking system for rotorcraft unmanned aerial vehicles," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, BC, Canada, 2017, pp. 1732-1738.
- [7] M.A. Awal, M.A.R. Refat, F. Naznin, M.Z. Islam, "A Particle Filter Based Visual Object Tracking: A Systematic Review of Current Trends and Research Challenges", *International Journal of Advanced Computer Science and Applications* Nov2023, Vol. 14 Issue 11, p1291-1301, 11p
- [8] G.M. Rao, C. Satyanarayana, "Visual Object Target Tracking Using Particle Filter: A Survey", *I.J. Image, Graphics and Signal Processing*, 2013, 6, 57-71
- [9] K. Nummiaro, E. Koller-Meier, and L. Van Gool, "Object Tracking with an Adaptive Color-Based Particle Filter" *Proceedings 2006 IEEE International Conference on Robotics and Automation*, 2006. ICRA 2006.
- [10] A. S. Sabbi and M. Huber, "Particle filter based object tracking in a stereo vision system," *Proceedings 2006 IEEE International Conference on Robotics and Automation*, 2006. ICRA 2006., Orlando, FL, USA, 2006, pp. 2409-2415
- [11] B. Stenger, A. Thayananthan, P. H. S. Torr and R. Cipolla, "Model-based hand tracking using a hierarchical Bayesian filter," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 9, pp. 1372-1384, Sept. 2006
- [12] Y. Dai, B. Liu, "Robust video object tracking via Bayesian model averaging-based feature fusion" *Opt.Eng.* 55(8), 083102 (2016)
- [13] Thrun, S. "Particle filters in robotics" *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, 2002 511-518.